Server Error Alerting — Approaches & Implementation Guide

Project: SkillSphere Backend (Express + MySQL + PM2 on cPanel) **Date:** 2026-10-24 **Purpose:** Get email notifications when the server crashes or a request fails, so you know before users complain.

The Problem We Are Solving

Right now when something breaks — like the OTP column being too small — it sits silently in PM2 logs. You only find out when a user messages you. This document covers three ways to fix that.

Approach 1: DIY Nodemailer Alerts (Recommended)

What It Does

Uses the same SMTP setup already in the project (`support@skillsphere.com.pk`) to send alert emails directly from the server code. No third-party accounts. No monthly cost.

What Triggers an Alert

Trigger	Throttle	Example
Server crash (`uncaughtException`)	None — always fires	Process about to die
Unhandled promise rejection	None — always fires	`await` without try/catch throwing
DB connection lost	None — always fires	MySQL goes down
Controller caught error	1 per error type per 15 min	OTP save failing, email not sending, payment error

What the Email Looks Like

```
Subject: [SkillSphere ALERT] SequelizeDatabaseError — authController.js

Environment : production
Time        : 2026-10-23 19:17:41 UTC
Function    : exports.sendLoginOTP (authController.js:488)

Error       : Data too long for column 'otpCode' at row 1

Stack Trace :
  at Query.run (.../sequelize/lib/dialects/mysql/query.js:52)
  at async exports.sendLoginOTP (authController.js:488)

SQL         : UPDATE users SET otpCode=?, otpExpiry=? WHERE id=?
Parameters  : ['562f70d9...', '2026-10-23 19:27:41', 2]
```

Files to Create / Change

```
backend/
  services/
    alertService.js        ← NEW: throttled email alert sender
  server.js                ← ADD: crash hooks (3 lines)
  controllers/
    authController.js      ← ADD: sendAlert() in catch blocks
    paymentController.js   ← ADD: sendAlert() in catch blocks
    enrollmentController.js← ADD: sendAlert() in catch blocks
```

How to Implement (Step by Step)

Step 1 — Add to `.env` :

```
ALERT_EMAILS=youremail@gmail.com,secondemail@gmail.com,thirdemail@gmail.com
```

Step 2 — Create `backend/services/alertService.js` :

```
const nodemailer = require('nodemailer');

const throttleMap = new Map(); // key -> last sent timestamp
const THROTTLE_MS = 15 * 60 * 1000; // 15 minutes

function shouldSend(key) {
  const last = throttleMap.get(key);
  if (!last) return true;
  return Date.now() - last > THROTTLE_MS;
}

async function sendAlert({ title, error, context = '' }) {
  const key = `${error.message?.slice(0, 60)}`;
  if (!shouldSend(key)) return; // throttled
  throttleMap.set(key, Date.now());

  const recipients = process.env.ALERT_EMAILS?.split(',') || [];
  if (!recipients.length) return;

  const transporter = nodemailer.createTransporter({
    host: process.env.SMTP_HOST,
    port: process.env.SMTP_PORT,
    auth: { user: process.env.SMTP_USER, pass: process.env.SMTP_PASS }
  });

  const body = `
Environment : ${process.env.NODE_ENV || 'unknown'}
Time        : ${new Date().toISOString()}
Context     : ${context}

Error       : ${error.message}
```

```
Stack Trace :
${error.stack}
  `.trim();

  await transporter.sendMail({
    from: process.env.SMTP_FROM_EMAIL,
    to: recipients.join(', '),
    subject: `[SkillSphere ALERT] ${title || error.name}`,
    text: body
  });
}

module.exports = { sendAlert };
```

Step 3 — Add crash hooks to `backend/server.js` :

```
const { sendAlert } = require('./services/alertService');

process.on('uncaughtException', async (error) => {
  await sendAlert({ title: 'CRASH — uncaughtException', error, context: 'Process will exit'
});
  process.exit(1);
});

process.on('unhandledRejection', async (reason) => {
  await sendAlert({ title: 'unhandledRejection', error: reason, context: 'Unhandled promise'
});
});
```

Step 4 — Add to controller catch blocks (example):

```
// In authController.js sendLoginOTP catch:
} catch (error) {
  sendAlert({ title: 'sendLoginOTP failed', error, context: 'POST /auth/login-otp' });
  console.error('Send Login OTP error:', error);
  return res.status(500).json({ error: 'Failed to send OTP' });
}
```

Pros

- Zero new dependencies (Nodemailer already installed)
- No monthly cost
- Full control over what triggers an alert
- Works offline from any third-party service

Cons

- If the SMTP server itself is down, alerts won't send
- You maintain it yourself
- In-memory throttle resets on PM2 restart (minor — could fire again after restart)

Approach 2: Sentry (Third-Party Error Tracking)

What It Does

Sentry is a dedicated error monitoring service. You install their SDK, it automatically wraps your entire Express app, catches every error (including ones you forgot to handle), and sends them to a web dashboard. You can also set up email/Slack notifications from the dashboard.

What You Get

- Web dashboard at `sentry.io` showing all errors, frequency, affected users
- Automatic stack traces with source maps
- Error grouping — same error grouped into one issue
- Email alerts per issue (configurable)
- Free tier: 5,000 errors/month

How to Implement

Step 1 — Create free account at sentry.io

Step 2 — Install:

```
npm install @sentry/node
```

Step 3 — Add to top of `server.js`:

```
const Sentry = require('@sentry/node');

Sentry.init({
  dsn: 'https://YOUR_DSN@sentry.io/PROJECT_ID',
  environment: process.env.NODE_ENV,
  tracesSampleRate: 0.1,
});

app.use(Sentry.Handlers.requestHandler());
// ... all your routes ...
app.use(Sentry.Handlers.errorHandler()); // must be after routes
```

Step 4 — Configure email alerts in Sentry dashboard:

- Go to Project Settings → Alerts
- Add your 3 email addresses as team members
- Set rule: "When a new issue is seen → email all members"

Pros

- Catches errors you forgot to handle automatically
- Rich dashboard — see trends, frequency, which users are affected
- No SMTP dependency for alerts
- Works even if your own mail server is down

Cons

- Requires a sentry.io account (free tier has limits)

- Adds an external dependency
- Data goes to a third-party server (GDPR consideration)
- Free tier: 5,000 errors/month — above that you pay
- Overkill for a small app

Approach 3: PM2 Process Hooks Only

What It Does

PM2 has built-in event hooks. You can configure it to run a script or send a notification when the process restarts or crashes. This only covers crashes — not individual request errors.

How to Implement

Option A — PM2 `ecosystem.config.js`:

```
module.exports = {
  apps: [{
    name: 'skillsphere',
    script: 'server.js',
    cwd: '/home/skillsph/skill-sphere-Cpanel/backend',
    env_production: { NODE_ENV: 'production' },
    // Run a script when PM2 restarts the app
    post_update: ['node notify-crash.js'],
  }]
};
```

Option B — `pm2-slack` or `pm2-discord` modules:

```
pm2 install pm2-slack
pm2 set pm2-slack:slack_url https://your-slack-webhook
```

What It Catches

- Process crashes (exit code non-zero)
- Process restart loops
- Does NOT catch: caught errors, DB failures, OTP failures, email failures

Pros

- No code changes in your app at all
- PM2 already manages your process

Cons

- Only catches crashes, not operational failures
- The OTP issue (which was a caught error) would NOT trigger this
- Slack webhook required for most integrations — no direct email

---

Comparison Summary

	DIY Nodemailer	Sentry	PM2 Hook
Setup effort	Medium (2-3 hours)	Low (30 min)	Very low (15 min)
Catches crashes	Yes	Yes	Yes
Catches request errors	Yes (where you add it)	Yes (automatic)	No
Catches the OTP bug	Yes	Yes	No
Cost	Free	Free up to 5k/mo	Free
External dependency	No	Yes (sentry.io)	No
Email delivery	Your SMTP	Sentry's servers	No direct email
Maintenance	You	Sentry	PM2 team
Dashboard	No	Yes	No

Recommendation

Go with Approach 1 (DIY Nodemailer) for your current scale.

- You already have Nodemailer and SMTP working
- No new accounts or external services
- Catches both crashes AND operational errors like the OTP bug
- The implementation is under 50 lines of code
- If your app grows significantly, migrate to Sentry later — the two are not mutually exclusive

The only risk is if your SMTP server goes down at the same time as the error. That is unlikely and acceptable for a small production app.

After You Decide

Once you pick an approach, the implementation is:

1. Update `.env` on the server
2. Upload 1-2 changed files
3. `pm2 restart skillsphere`

No DB changes required for any approach.